

Guide d'Accompagnement du TP Avancé : Architectures Web Modernes avec Supabase

Module : Architecture Web – Niveau Bac+4

Enseignant : BOURGUIBA

Table des matières

1	Objectif du TP	2
2	Architecture Client-Serveur avec Supabase	3
2.1	Création du projet Supabase	3
2.2	Création de la base de données	3
2.3	Création du front-end React	3
2.4	Tests et Sécurité	4
3	Architecture Microservices	5
3.1	Objectif	5
3.2	Préparation de l'environnement	5
3.3	Auth Service (exemple)	5
3.4	Dockerfile	5
3.5	Docker Compose	6
3.6	Sécurité	6
4	Architecture Décentralisée (DApp)	7
4.1	Objectif	7
4.2	Environnement	7
4.3	Smart Contract	7
4.4	Interface Web	7
5	Annexes – Explications des notions clés	8
6	Résultat attendu	8

1 Objectif du TP

Ce document a pour objectif de guider pas à pas la réalisation du projet **SafeDocs**, une application de gestion sécurisée de fichiers.

Les étudiants expérimenteront trois types d'architectures :

1. Architecture **Client-Serveur** classique
2. Architecture **Microservices**
3. Application **Décentralisée (DApp)**

Les principales technologies utilisées sont :

- **Supabase** pour la base de données, l'authentification et le stockage
- **Docker** pour la conteneurisation des microservices
- **Ethereum** / **Remix IDE** / **MetaMask** pour la DApp

2 Architecture Client-Serveur avec Supabase

2.1 Création du projet Supabase

1. Se connecter à <https://supabase.com>
2. Créer un nouveau projet : **SafeDocs-CS**
3. Noter les identifiants **SUPABASE_URL** et **ANON_KEY**

Supabase crée automatiquement :

- Une base **PostgreSQL**
- Un **API REST**
- Un système d'**authentification**
- Un **stockage de fichiers**

2.2 Création de la base de données

Dans l'onglet Table Editor, créer la table **files** avec les colonnes suivantes :

Colonne	Type
id	uuid (Primary key)
user_id	uuid
filename	text
url	text
created_at	timestamp

Activer les **Row Level Security (RLS)** et ajouter une règle :

```
USING (auth.uid() = user_id);
```

2.3 Création du front-end React

```
npx create-react-app safedocs-client
cd safedocs-client
npm install @supabase/supabase-js
```

Client Supabase :

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = 'https://votre-url.supabase.co';
const supabaseKey = 'votre-anon-key';
export const supabase = createClient(supabaseUrl, supabaseKey);
```

Fonctionnalités principales :

```
// Authentification
await supabase.auth.signUp({ email, password });
await supabase.auth.signInWithPassword({ email, password });

// Upload de fichier
await supabase.storage.from('docs').upload(`/${user.id}/${file.name}`,
  ↪ file);

// Lecture des fichiers
const { data } = await supabase.from('files').select('*');
```

2.4 Tests et Sécurité

- Test des routes via **Postman**
- Vérification du RLS via la console Supabase
- Scan de sécurité rapide avec **OWASP ZAP**

Objectif atteint : une application client-serveur sécurisée avec Supabase.

3 Architecture Microservices

3.1 Objectif

Transformer l'application précédente en trois microservices indépendants :

Service	Rôle	Description
Auth Service	Authentification	Gère les utilisateurs via Supabase
File Service	Fichiers	Gère l'accès au stockage
Gateway	Point d'entrée	Redirige les requêtes et sécurise

3.2 Préparation de l'environnement

1. Installer Docker Desktop
2. Créer un dossier :

```
safedocs-micro/  
    auth-service/  
    file-service/  
    gateway/
```

3.3 Auth Service (exemple)

```
import express from 'express';  
import { createClient } from '@supabase/supabase-js';  
  
const app = express();  
app.use(express.json());  
const supabase = createClient(process.env.SUPABASE_URL, process.env.  
    ↪ SUPABASE_KEY);  
  
app.post('/login', async (req, res) => {  
    const { email, password } = req.body;  
    const { data, error } = await supabase.auth.signInWithPassword({  
        ↪ email, password });  
    if (error) return res.status(400).json(error);  
    res.json(data);  
});  
  
app.listen(3001, () => console.log('Auth service running on port 3001'))  
    ↪ );
```

3.4 Dockerfile

```
FROM node:18  
WORKDIR /app  
COPY . .  
RUN npm install  
CMD ["node", "index.js"]
```

3.5 Docker Compose

```
version: "3"
services:
  auth:
    build: ./auth-service
    ports:
      - "3001:3001"
  file:
    build: ./file-service
    ports:
      - "3002:3002"
  gateway:
    build: ./gateway
    ports:
      - "8080:8080"
```

Lancer l'ensemble :

```
docker-compose up --build
```

Résultat : chaque service fonctionne indépendamment.

3.6 Sécurité

- Utilisation de **JWT** pour sécuriser les requêtes
- Mise en place d'un proxy HTTPS (bonus)
- Tests via Postman

4 Architecture Décentralisée (DApp)

4.1 Objectif

Créer une version décentralisée de SafeDocs utilisant la blockchain Ethereum.

4.2 Environnement

- Installer **MetaMask**
- Utiliser le **réseau de test Sepolia**
- Ouvrir **Remix IDE** : <https://remix.ethereum.org>

4.3 Smart Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SafeDocs {
    struct File {
        string name;
        string ipfsHash;
    }

    mapping(address => File[]) public files;

    function addFile(string memory _name, string memory _ipfsHash)
        ↪ public {
        files[msg.sender].push(File(_name, _ipfsHash));
    }

    function getFiles() public view returns (File[] memory) {
        return files[msg.sender];
    }
}
```

4.4 Interface Web

```
npm install ethers
```

```
const provider = new ethers.BrowserProvider(window.ethereum);
const contract = new ethers.Contract(contractAddress, abi, await
    ↪ provider.getSigner());
```

Uploader les fichiers sur IPFS via <https://web3.storage>

Résultat : une application décentralisée de stockage de documents.

5 Annexes – Explications des notions clés

- **Supabase** : plateforme open-source offrant PostgreSQL, API REST, Auth et stockage.
- **Microservices** : architecture découpant une application en services indépendants.
- **API Gateway** : point d'entrée unique des microservices, gérant sécurité et routage.
- **Docker** : outil de conteneurisation isolant les services dans des environnements distincts.
- **DApp** : application décentralisée fonctionnant sur la blockchain.
- **Smart Contract** : code auto-exécuté sur la blockchain contenant la logique métier.
- **IPFS** : système de stockage distribué sans serveur central.
- **JWT** : token d'authentification signé utilisé entre services.

6 Résultat attendu

À la fin du TP, l'étudiant devra être capable de :

- Déployer une application client-serveur sécurisée avec Supabase
- Transformer cette application en architecture microservices
- Créer une version décentralisée (DApp)
- Rédiger un comparatif technique entre les trois architectures